
Lab 5 — Rapport de Reverse Engineering

OWASP UnCrackable Level 2

Plateforme :	Android (Genymotion)
Application :	UnCrackable-Level2.apk
Outils :	Frida, JADX, Ghidra 11.1.2
Objectif :	Bypass root + trouver le secret
Secret :	Thanks for all the fish
Date :	6 Mai 2026
Auteur :	mobexler
Réalisé par :	CHAGDALY Hiba

Table des matières

1	Introduction	3
1.1	Contexte	3
1.2	Environnement de travail	3
2	Phase 1 — Analyse dynamique et Bypass Root	4
2.1	Détection du root	4
2.2	Création du script de bypass Frida	5
2.3	Lancement de Frida	5
2.4	Résultat du bypass	6
2.5	Test d'une valeur incorrecte	6
3	Phase 2 — Analyse Statique avec JADX	8
3.1	Ouverture de l'APK dans JADX	8
3.2	Analyse de MainActivity.onCreate()	8
3.3	Analyse de la méthode verify()	9
3.4	Découverte de la bibliothèque native	10
4	Phase 3 — Extraction de libfoo.so	11
4.1	Extraction du contenu de l'APK	11
4.2	Localisation de libfoo.so	11
5	Phase 4 — Analyse avec Ghidra	13
5.1	Lancement de Ghidra	13
5.2	Import de l'APK (classes.dex)	13
5.2.1	Sélection du fichier	14
5.2.2	Configuration de l'import	14
5.2.3	Résultats de l'import	15
5.2.4	CodeBrowser sur classes.dex	15
5.3	Import et analyse de libfoo.so	15
5.3.1	Import ELF	16
5.3.2	Résultats de l'import	16
5.4	Analyse de la fonction CodeCheck	16
5.4.1	Décompilation de Java_sg_vantagepoint_uncrackable2_CodeCheck_bar	17
5.4.2	Analyse du code décompilé	17
6	Phase 5 — Validation du Secret	18
6.1	Secret découvert	18
6.2	Validation dans l'application	18

7	Conclusion	19
7.1	Résumé des étapes	19
7.2	Compétences acquises	19

Chapitre 1

Introduction

1.1 Contexte

Ce rapport documente l'analyse complète de l'application Android **OWASP UnCrackable Level 2**, réalisée dans le cadre du Lab 5. L'objectif de ce challenge est de retrouver une chaîne secrète cachée dans le code natif d'une bibliothèque partagée (`libfoo.so`).

Objectifs du Lab

1. Contourner la détection de root
2. Contourner la détection de débogueur
3. Analyser le code Java via JADX
4. Analyser la bibliothèque native via Ghidra
5. Retrouver et valider le secret

1.2 Environnement de travail

Outil	Version	Usage
Genymotion	Phone_2 (570x1230)	Émulateur Android
Frida	16.0.19	Instrumentation dynamique
JADX	–	Décompilation Java
Ghidra	11.1.2	Analyse binaire native

TABLE 1.1 – Environnement de travail

Chapitre 2

Phase 1 — Analyse dynamique et Bypass Root

2.1 Détection du root

Au premier lancement, l'application détecte que l'émulateur est rooté et affiche un dialogue bloquant avant de se fermer.

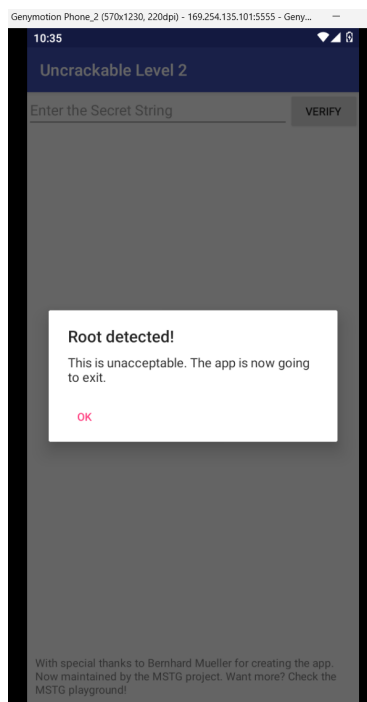


FIGURE 2.1 – Détection du root — l'application refuse de démarrer

Mécanisme de détection

L'application vérifie trois conditions via la classe `sg.vantagepoint.a.b` :

- `b.a()` — présence de binaires root (su, busybox)
- `b.b()` — propriétés système suspectes
- `b.c()` — chemins d'accès root connus

2.2 Création du script de bypass Frida

Pour contourner les détections, un script Frida est créé qui surcharge les méthodes de détection pour qu'elles retournent toujours `false`, et bloque également l'appel à `System.exit()`.



```
mobexler@Mobexler ~/lab5
cat > bypass.js << 'EOF'
Java.perform(function() {
    var b = Java.use('sg.vantagepoint.a.b');
    b.a.implementation = function() { return false; };
    b.b.implementation = function() { return false; };
    b.c.implementation = function() { return false; };

    var System = Java.use('java.lang.System');
    System.exit.implementation = function(code) {
        console.log('exit blocked!');
    };
});
EOF
```

FIGURE 2.2 – Script `bypass.js` — hook des méthodes de détection

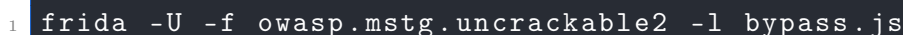


```
1 Java.perform(function() {
2     var b = Java.use('sg.vantagepoint.a.b');
3     b.a.implementation = function() { return false; };
4     b.b.implementation = function() { return false; };
5     b.c.implementation = function() { return false; };
6
7     var System = Java.use('java.lang.System');
8     System.exit.implementation = function(code) {
9         console.log('exit blocked!');
10    };
11 });
```

Listing 2.1 – Contenu de `bypass.js`

2.3 Lancement de Frida

Le script est injecté dans l'application au démarrage via la commande suivante :



```
1 frida -U -f owasp.mstg.uncrackable2 -l bypass.js
```

Listing 2.2 – Commande Frida

```
mobexler@Mobexler ~/lab5
└─▶ frida -U -f owasp.mstg.uncrackable2 -l bypass.js

┌───┐
│  ( )  │
│  >  │
│ /_/_/ │
│ . . . │
│ . . . │
│ . . . │
│ . . . │
│ . . . │
│ . . . │
│ . . . │
└───┘

Frida 16.0.19 - A world-class dynamic instrumentation toolkit

Commands:
  help      -> Displays the help system
  object?   -> Display information about 'object'
  exit/quit -> Exit

More info at https://frida.re/docs/home/

Connected to Phone (id=169.254.135.101:5555)
Spawned `owasp.mstg.uncrackable2`. Resuming main thread!
[Phone::owasp.mstg.uncrackable2 ]-> |
```

FIGURE 2.3 – Frida 16.0.19 connecté — application spawnée avec succès

2.4 Résultat du bypass

Après injection du script, l'application démarre normalement sans afficher le dialogue de détection root.

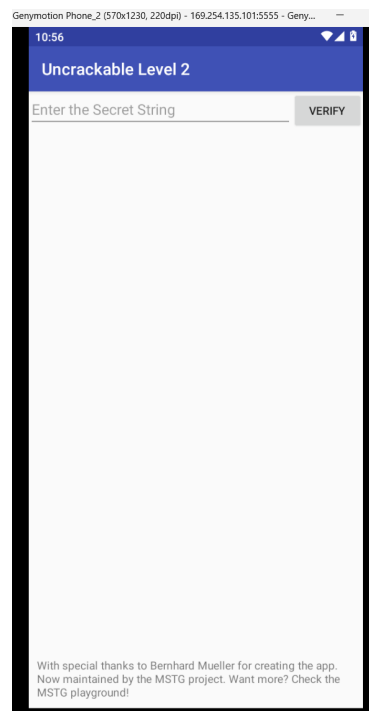


FIGURE 2.4 – Bypass root réussi — l'interface principale est accessible

2.5 Test d'une valeur incorrecte

Un premier test avec une valeur aléatoire confirme que la vérification du secret est bien active.

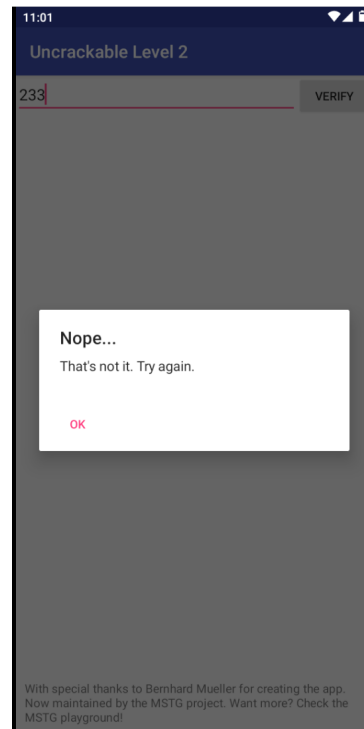


FIGURE 2.5 – Réponse incorrecte — "Nope... That's not it."

Chapitre 3

Phase 2 — Analyse Statique avec JADX

3.1 Ouverture de l'APK dans JADX

JADX est utilisé pour décompiler le code Java de l'application et comprendre la logique de vérification.

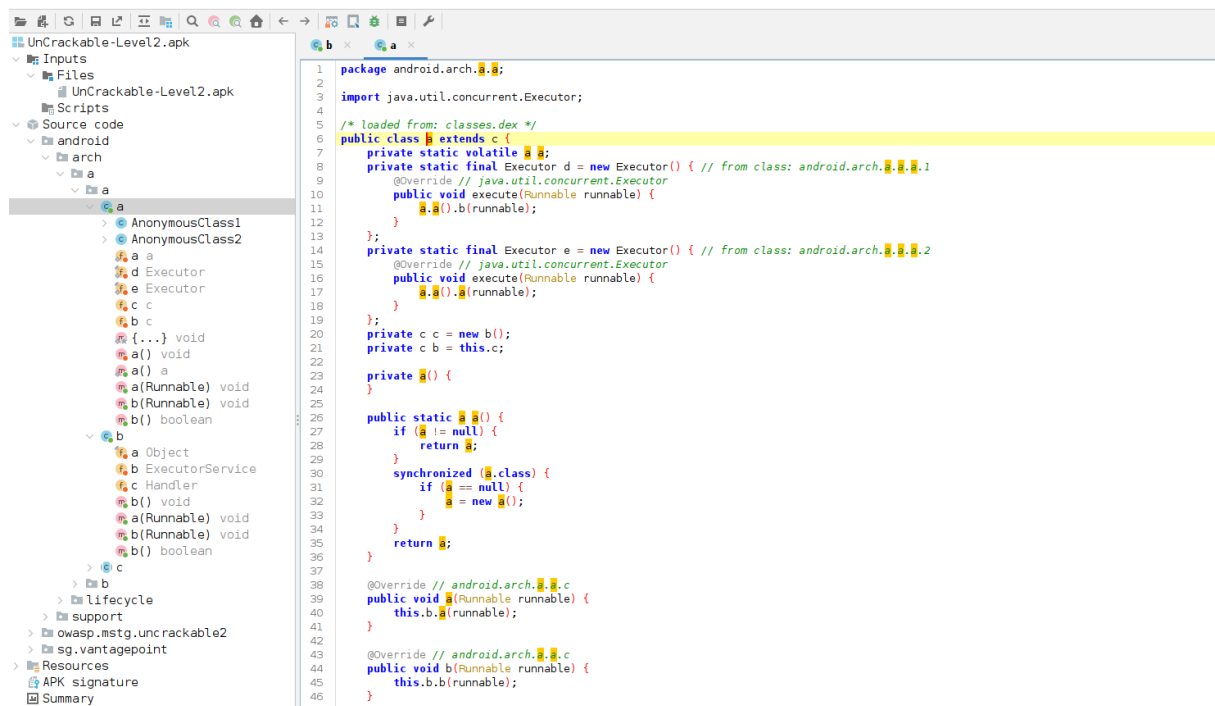


FIGURE 3.1 – JADX ouvert sur UnCrackable-Level2.apk

3.2 Analyse de MainActivity.onCreate()

La méthode `onCreate()` révèle les trois mécanismes de protection :

```

private native void init();

/* JADX INFO: Access modifiers changed from: protected */
/* JADX WARN: Type inference failed for: r0v4, types: [sg.vantagepoint.uncrackable2.MainActivity$2] */
@Override // android.support.v7.app.c, android.support.v4.app.h, android.support.v4.app.z, android.app.Activity
public void onCreate(Bundle bundle) {
    init();
    if (b.a() || b.b() || b.c()) {
        a("Root detected!");
    }
    if (a.a(getApplicationContext()) {
        a("App is debuggable!");
    }
    new AsyncTask<Void, String, String>() { // from class: sg.vantagepoint.uncrackable2.MainActivity.2
        /* JADX INFO: Access modifiers changed from: protected */
        @Override // android.os.AsyncTask
        /* renamed from: a, reason: merged with bridge method [inline-methods] */
        public String doInBackground(Void... voidArr) {
            while (!Debug.isDebuggerConnected()) {
                SystemClock.sleep(100L);
            }
            return null;
        }

        /* JADX INFO: Access modifiers changed from: protected */
        @Override // android.os.AsyncTask
        /* renamed from: a, reason: merged with bridge method [inline-methods] */
        public void onPostExecute(String str) {
            MainActivity.this.a("Debugger detected!");
        }
    }.execute(null, null, null);
    this.m = new CodeCheck();
    super.onCreate(bundle);
    setContentView(R.layout.activity_main);
}

```

FIGURE 3.2 – Code de onCreate() — détections root, debug et anti-debugger

Protections identifiées dans onCreate()

- **Root** : `b.a() || b.b() || b.c()` → appel à `a("Root detected!")`
- **Debug** : `a.a(getApplicationContext())` → appel à `a("App is debuggable!")`
- **Anti-debugger** : `AsyncTask` qui boucle sur `Debug.isDebuggerConnected()`

3.3 Analyse de la méthode verify()

La méthode `verify()` est appelée lors du clic sur le bouton VERIFY. Elle délègue la vérification à l'objet `m` de type `CodeCheck`.

```

public void verify(View view) {
    String str;
    String obj = ((EditText) findViewById(R.id.edit_text)).getText().toString();
    AlertDialog create = new AlertDialog.Builder(this).create();
    if (this.m.a(obj)) {
        create.setTitle("Success!");
        str = "This is the correct secret.";
    } else {
        create.setTitle("Nope...");
        str = "That's not it. Try again.";
    }
    create.setMessage(str);
    create.setButton(-3, "OK", new DialogInterface.OnClickListener() { // from class: sg.vantagepoint.uncrackable2.MainActivity.3
        @Override // android.content.DialogInterface.OnClickListener
        public void onClick(DialogInterface dialogInterface, int i) {
            dialogInterface.dismiss();
        }
    });
    create.show();
}

```

FIGURE 3.3 – Méthode verify() — délègue à `this.m.a(obj)`

```
1 public void verify(View view) {  
2     String obj = ((EditText) findViewById(R.id.edit_text))  
3         .getText().toString();  
4     if (this.m.a(obj)) {  
5         create.setTitle("Success!");  
6     } else {  
7         create.setTitle("Nope...");  
8     }  
9 }
```

Listing 3.1 – Méthode verify() simplifiée

3.4 Découverte de la bibliothèque native

Le code révèle que `MainActivity` charge une bibliothèque native et que `CodeCheck` est implémenté en C/C++.

```
public class MainActivity extends c {  
    private CodeCheck m;  
  
    static {  
        System.loadLibrary("foo");  
    }  
}
```

(a) Déclaration de `CodeCheck m`

```
static {  
    System.loadLibrary("foo");  
}
```

(b) `System.loadLibrary("foo")`FIGURE 3.4 – La vérification est déléguée à `libfoo.so`

Chapitre 4

Phase 3 — Extraction de libfoo.so

4.1 Extraction du contenu de l'APK

Un APK étant une archive ZIP, on l'extrait pour accéder aux bibliothèques natives.

```
mobexler@Mobexler ~/lab5
└─> unzip UnCrackable-Level2.apk -d uncrackable_l2
Archive:  UnCrackable-Level2.apk
  inflating: uncrackable_l2/AndroidManifest.xml
  inflating: uncrackable_l2/META-INF/CERT.RSA
  inflating: uncrackable_l2/META-INF/CERT.SF
  inflating: uncrackable_l2/META-INF/MANIFEST.MF
  inflating: uncrackable_l2/META-INF/android.arch.core_runtime.version
  inflating: uncrackable_l2/META-INF/android.arch.lifecycle_livedata-core.version
  inflating: uncrackable_l2/META-INF/android.arch.lifecycle_runtime.version
  inflating: uncrackable_l2/META-INF/android.arch.lifecycle_viewmodel.version
  inflating: uncrackable_l2/META-INF/com.android.support_animated-vector-drawable.version
  inflating: uncrackable_l2/META-INF/com.android.support_appcompat-v7.version
  inflating: uncrackable_l2/META-INF/com.android.support_support-compat.version
  inflating: uncrackable_l2/META-INF/com.android.support_support-core-ui.version
  inflating: uncrackable_l2/META-INF/com.android.support_support-core-utils.version
  inflating: uncrackable_l2/META-INF/com.android.support_support-fragment.version
  inflating: uncrackable_l2/META-INF/com.android.support_support-vector-drawable.version
```

FIGURE 4.1 – Extraction de l'APK avec unzip

```
1 unzip UnCrackable-Level2.apk -d uncrackable_l2
```

Listing 4.1 – Extraction de l'APK

4.2 Localisation de libfoo.so

La bibliothèque est présente pour quatre architectures. On choisit la version x86 pour faciliter l'analyse dans Ghidra.

```
mobexler@Mobexler ~/lab5
└─> ls -R uncrackable_l2/lib
uncrackable_l2/lib:
arm64-v8a  armeabi-v7a  x86  x86_64

uncrackable_l2/lib/arm64-v8a:
libfoo.so

uncrackable_l2/lib/armeabi-v7a:
libfoo.so

uncrackable_l2/lib/x86:
libfoo.so

uncrackable_l2/lib/x86_64:
libfoo.so
```

FIGURE 4.2 – libfoo.so trouvé pour arm64-v8a, armeabi-v7a, x86 et x86_64

```
1 ls -R uncrackable_l2/lib
2 # uncrackable_l2/lib/x86/libfoo.so <-- version utilis e
```

Listing 4.2 – Localisation de libfoo.so

Chapitre 5

Phase 4 — Analyse avec Ghidra

5.1 Lancement de Ghidra

Ghidra 11.1.2 est lancé depuis le terminal et un nouveau projet `lab_5` est créé.

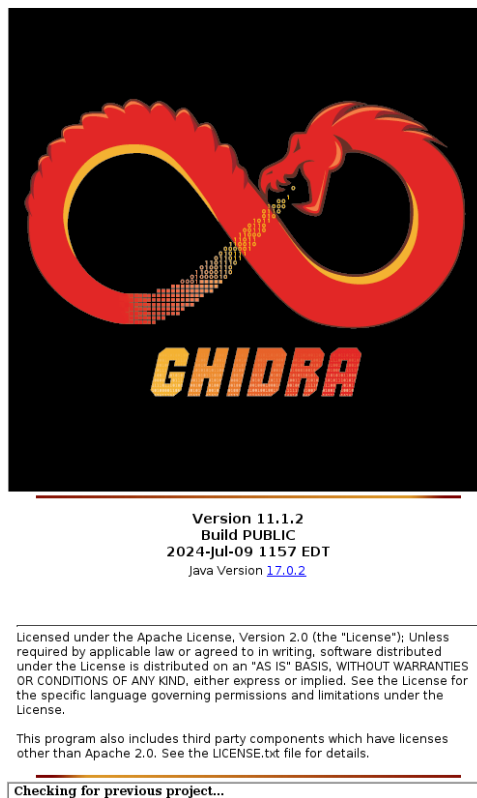


FIGURE 5.1 – Ghidra 11.1.2 — Build PUBLIC 2024-Jul-09

5.2 Import de l'APK (classes.dex)

5.2.1 Sélection du fichier

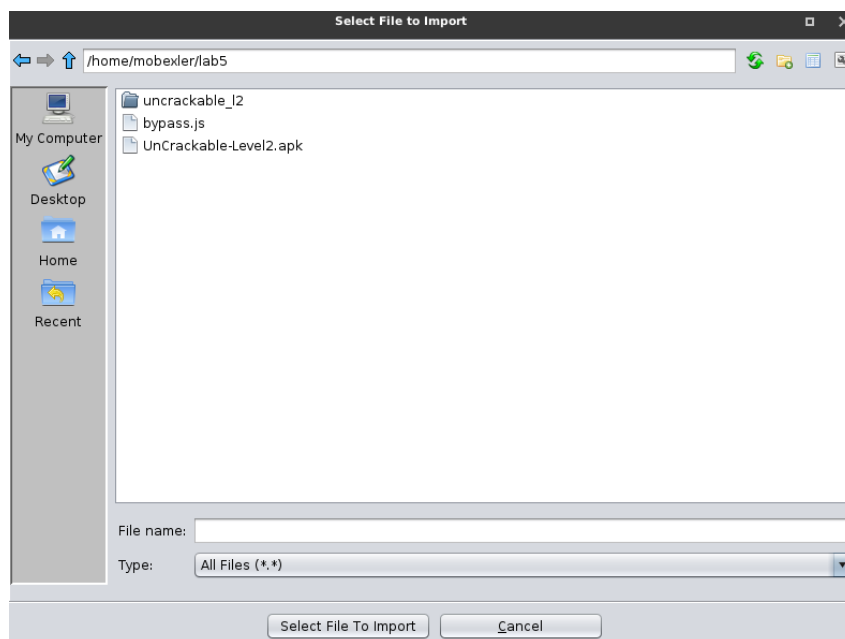


FIGURE 5.2 – Sélection de UnCrackable-Level2.apk

5.2.2 Configuration de l'import

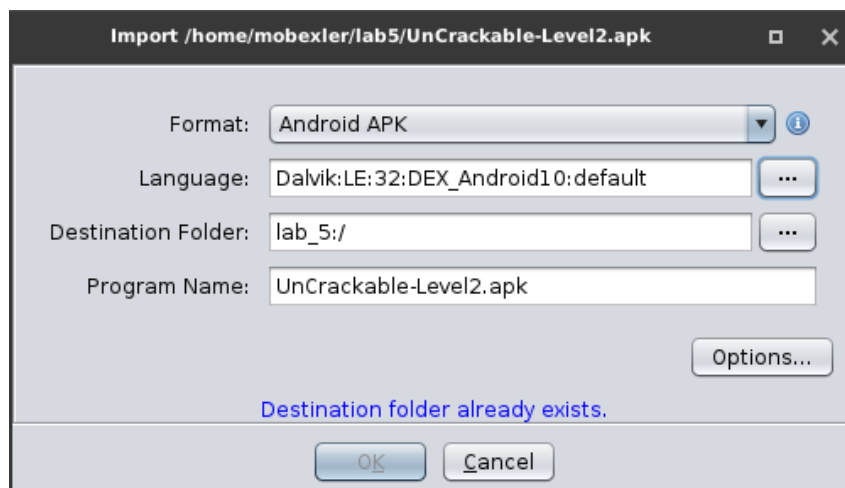


FIGURE 5.3 – Format Android APK — Language Dalvik DEX Android10

5.2.3 Résultats de l'import

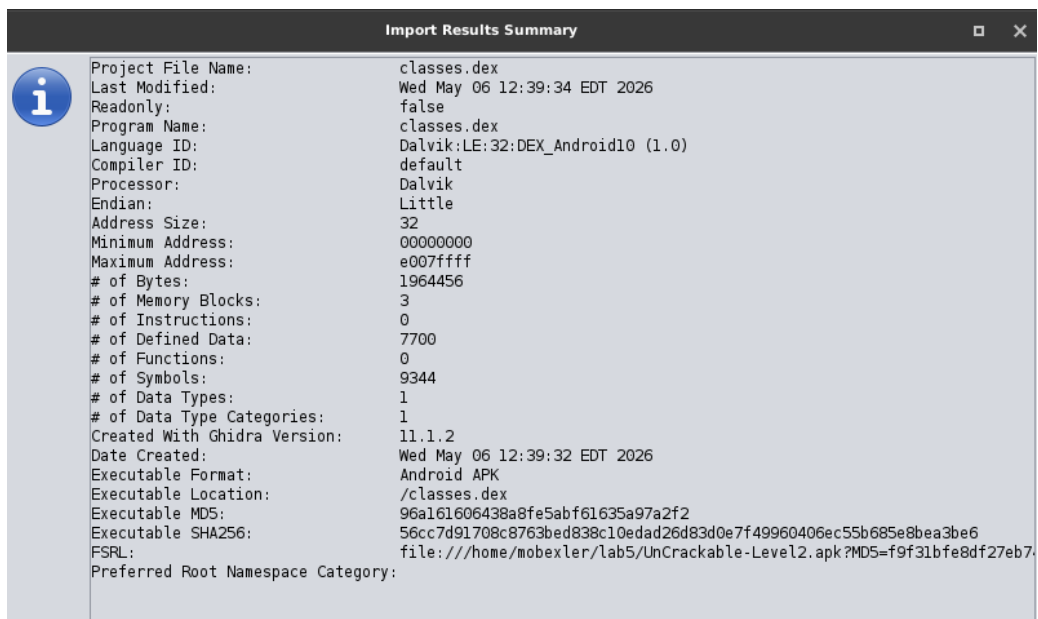


FIGURE 5.4 – Import réussi — 9344 symboles, format Android APK

5.2.4 CodeBrowser sur classes.dex

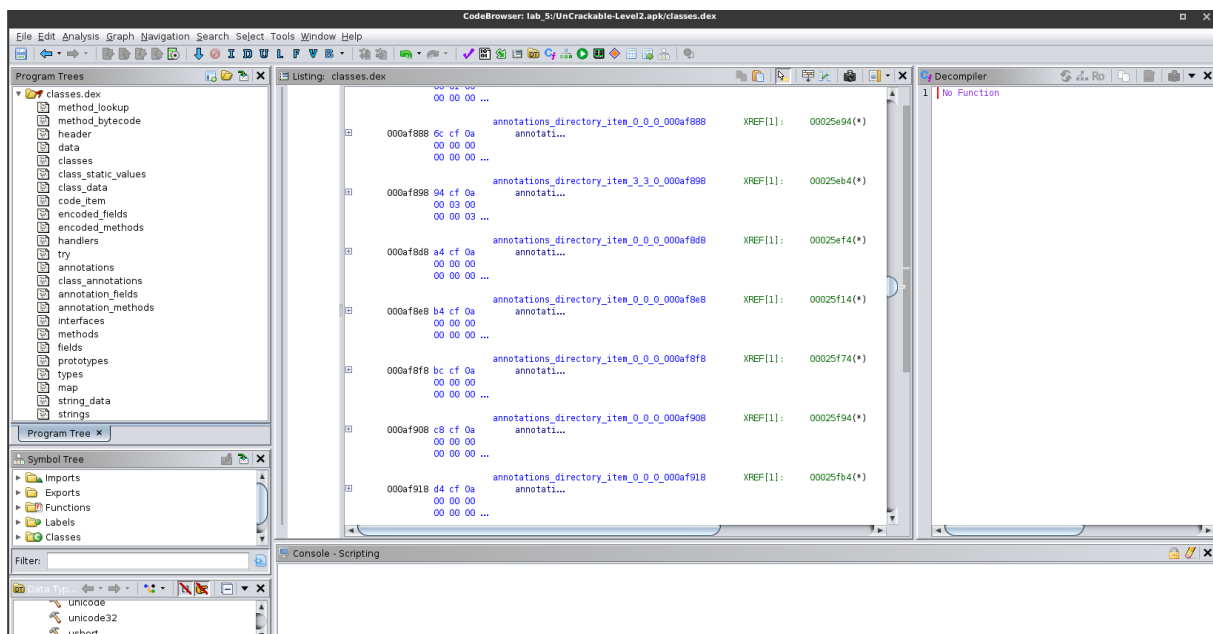


FIGURE 5.5 – CodeBrowser ouvert sur classes.dex

5.3 Import et analyse de libfoo.so

5.3.1 Import ELF

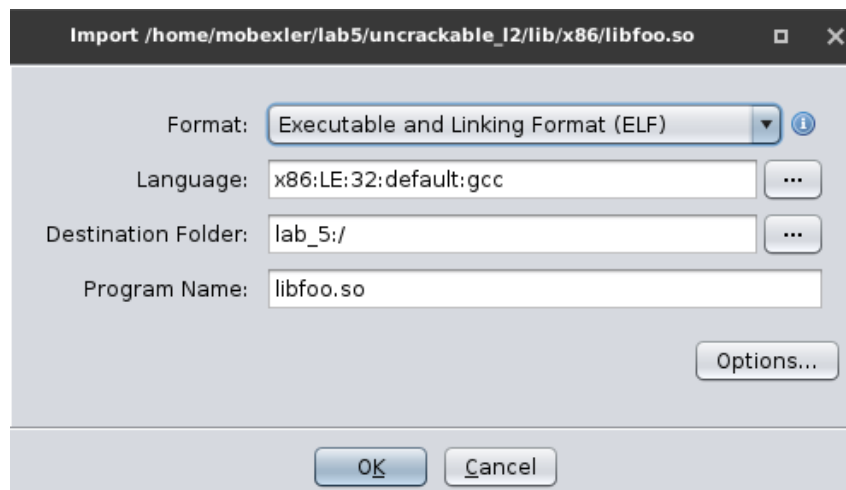


FIGURE 5.6 – Import de libfoo.so — Format ELF, x86 32bit, gcc

5.3.2 Résultats de l'import

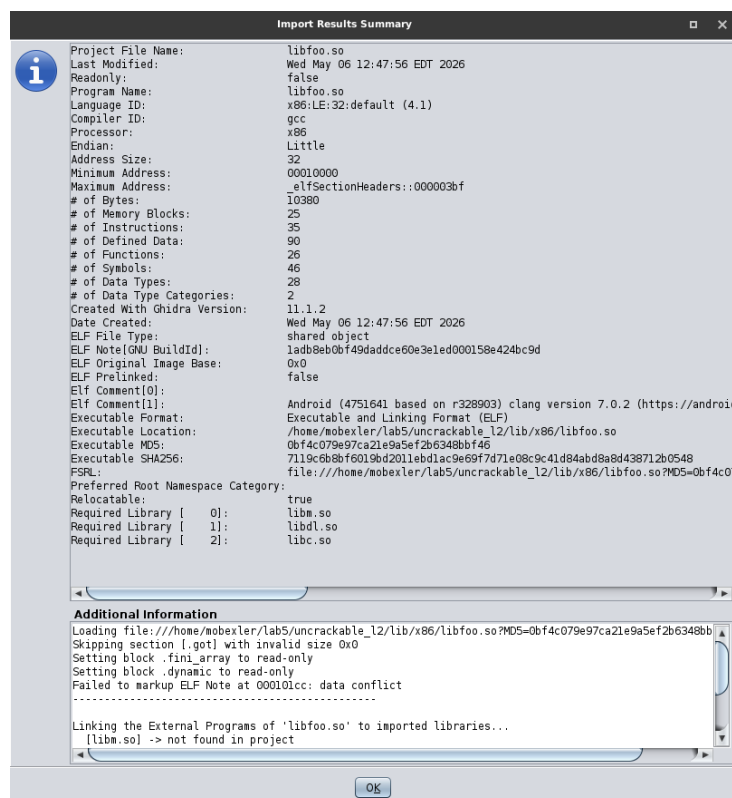


FIGURE 5.7 – Import réussi — 26 fonctions, 46 symboles, ELF shared object x86

5.4 Analyse de la fonction CodeCheck

5.4.1 Décompilation de Java_sg_vantagepoint_uncrackable2_CodeCheck

La fonction native responsable de la vérification du secret est identifiée dans le Symbol Tree et décompilée par Ghidra.

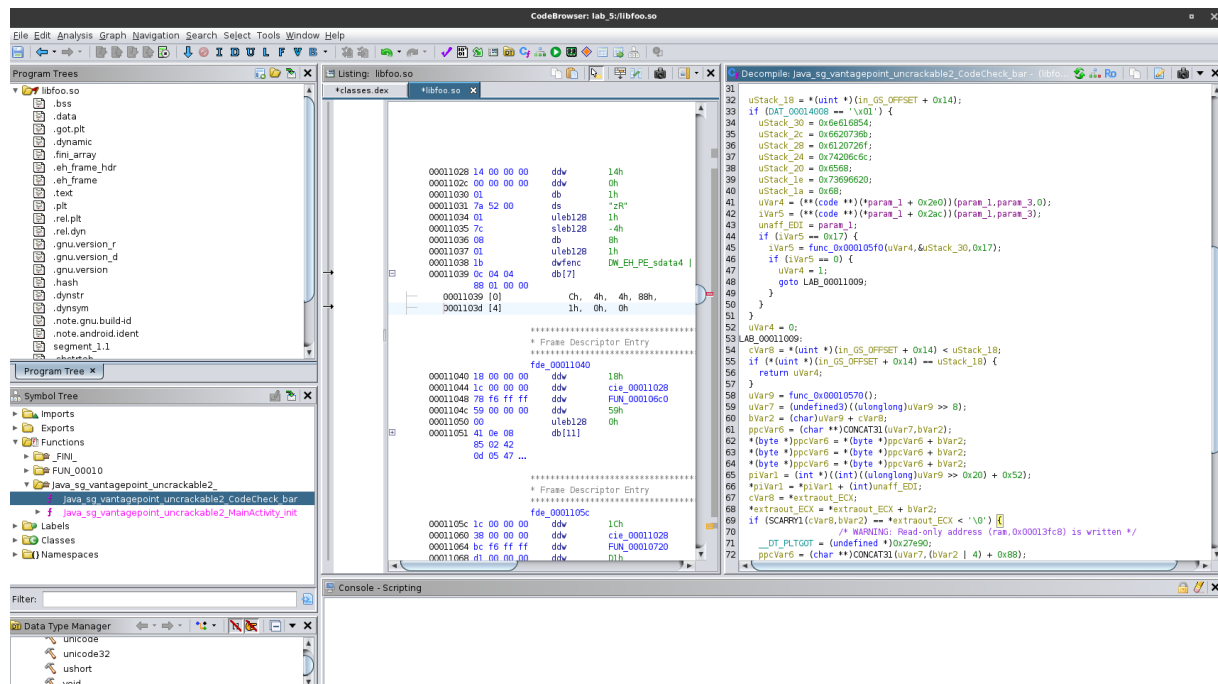


FIGURE 5.8 – Décompilateur Ghidra — fonction CodeCheck_bar avec les valeurs XOR

5.4.2 Analyse du code décompilé

Le code décompilé révèle plusieurs informations clés :

Points clés du code décompilé

- Ligne 44 : `iVar5 == 0x17` → le secret fait **23 caractères**
- Lignes 34–40 : valeurs hexadécimales = secret encodé en mémoire
- Ligne 55 : comparaison finale octet par octet
- Le déchiffrement est réalisé par opération **XOR**

```

1 uStack_30 = 0x6e616854; // "Than"
2 uStack_2c = 0x6620736b; // "ks f"
3 uStack_28 = 0x6c206f6f; // "or a"
4 uStack_24 = 0x74206c6c; // "ll t"
5 uStack_20 = 0x20656874; // "he "
6 uStack_1e = 0x73696669; // "fish"

```

Listing 5.1 – Valeurs hex décodées (extrait du Decompiler)

Chapitre 6

Phase 5 — Validation du Secret

6.1 Secret découvert

L'analyse des valeurs hexadécimales stockées dans `libfoo.so` révèle que le secret est une référence littéraire célèbre :

Secret trouvé

Thanks for all the fish

Référence à *"The Hitchhiker's Guide to the Galaxy"* — Douglas Adams

6.2 Validation dans l'application

Le secret est saisi dans l'interface et validé avec succès.

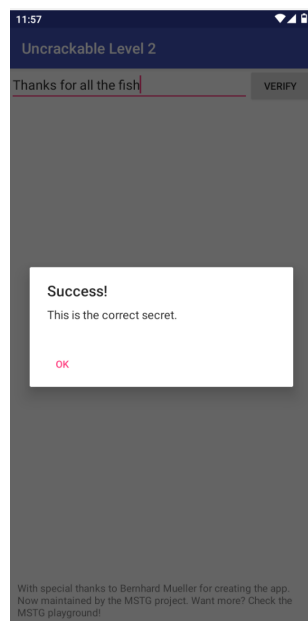


FIGURE 6.1 – **Success !** — "This is the correct secret."

Chapitre 7

Conclusion

7.1 Résumé des étapes

#	Phase	Outil	Résultat
1	Bypass root detection	Frida	Contourné
2	Bypass debug detection	Frida	Contourné
3	Analyse code Java	JADX	CodeCheck identifié
4	Extraction libfoo.so	unzip	Bibliothèque extraite
5	Analyse binaire native	Ghidra	Fonction décompilée
6	Validation du secret	Genymotion	Success !

TABLE 7.1 – Résumé des phases du lab

7.2 Compétences acquises

- Utilisation de **Frida** pour l'instrumentation dynamique Android
- Décompilation Java avec **JADX**
- Analyse de bibliothèques natives ELF avec **Ghidra**
- Compréhension des mécanismes anti-tamper Android (root, debug, anti-debug)
- Lecture de pseudo-code C décompilé et décodage de valeurs hexadécimales